Testes de Desempenho

Link Extractor

Realização de Testes de Carga com Múltiplas Configurações de API Python e Ruby com e sem Cache Redis

Edinei Xavier

2310369

Matheus Norões

2224600

Lucas Falcão

2315036

Samir Alves

2315046

A Aplicação Link Extractor

O que é a Aplicação?

Aplicação web que extrai e exibe todos os links encontrados em uma página a partir de uma URL. Desenvolvida como exemplo de microsserviços containerizados com Docker.

Arquitetura

Front-end em PHP — recebe a URL e exibe os links

Serviço de extração — Python/Ruby, faz o scraping

Redis — cache que armazena links extraídos

Fluxo com Cache

01 Front-end envia URL ao serviço

02 Serviço verifica presença no Redis

03 Se **SIM** → Retorna do cache (rápido)

04 Se **NÃO** → Extrai, salva no Redis e retorna

Ferramenta de Teste de Carga — Locust

O que é o Locust?

Ferramenta open-source de teste de carga escrita em Python.

Cada usuário virtual é uma corrotina Python que executa tarefas definidas na classe HttpUser.

Principais recursos utilizados:

- **SequentialTaskSet**

— executa as 10 URLs em ordem sequencial por usuário

- **--headless**

— modo sem interface, ideal para automação via terminal

- **--request-name**

— nomeia o cenário no relatório CSV

--csv — exporta resultados automaticamente

Script do usuário virtual

```python
class ExtracaoSequencial(SequentialTaskSet):

    links = [
        "https://unifor.br/web/graduacao/medicina",
        "https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/c/covid-19",
        "https://www.gov.br/pt-br",
        "https://g1.globo.com/sp/campinas-regiao/noticia/2026/05/09/produtos-ype-nao-recomendados-pela-
        "https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/h/hantavirose",
        "https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/a/aids-hiv",
        "https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/a/autismo",
        "https://www.gov.br/saude/pt-br/assuntos/saude-com-ciencia",
        "https://www.gov.br/saude/pt-br/assuntos/saude-de-a-a-z/s/sindrome-de-burnout",
        "https://g1.globo.com/saude/noticia/2026/05/09/brasil-casos-de-hantavirus-no-ano-sem-elo-com-ge
    ]


    @task
    def executar(self):
        request_name = self.user.environment.parsed_options.request_name

        for link in self.links:

            # encoded_link = quote(link, safe='')

            self.client.get(
                f"/api/{link}",
                name=request_name
            )

class WebsiteUser(HttpUser):
    tasks = [ExtracaoSequencial]
```

Cenários de Teste Avaliados

Python + Redis

API Python com cache Redis ativo. Respostas já processadas são retornadas diretamente do cache sem novo scraping.

Python sem Redis

API Python sem cache. Cada requisição faz scraping completo da URL em tempo real — o cenário mais custoso.

Ruby + Redis

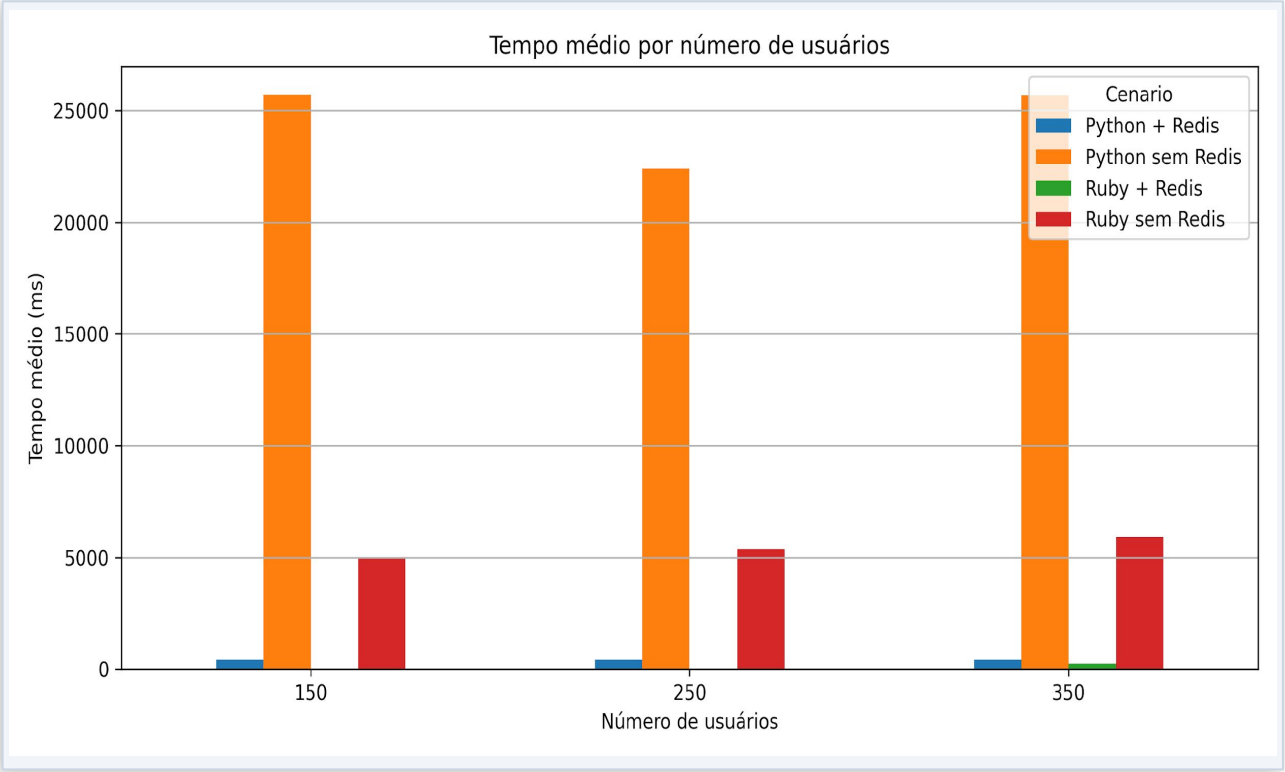
API Ruby com cache Redis. Mesmo benefício do cache, implementação em linguagem Ruby.

Ruby sem Redis

API Ruby sem cache. Scraping completo a cada requisição, equivalente ao Python sem Redis.

3 níveis de carga: Leve (150u) • Médio (250u) • Pesado (350u) 10 URLs sequenciais por usuário • Spawn rate: 5/s • Duração: 1 min

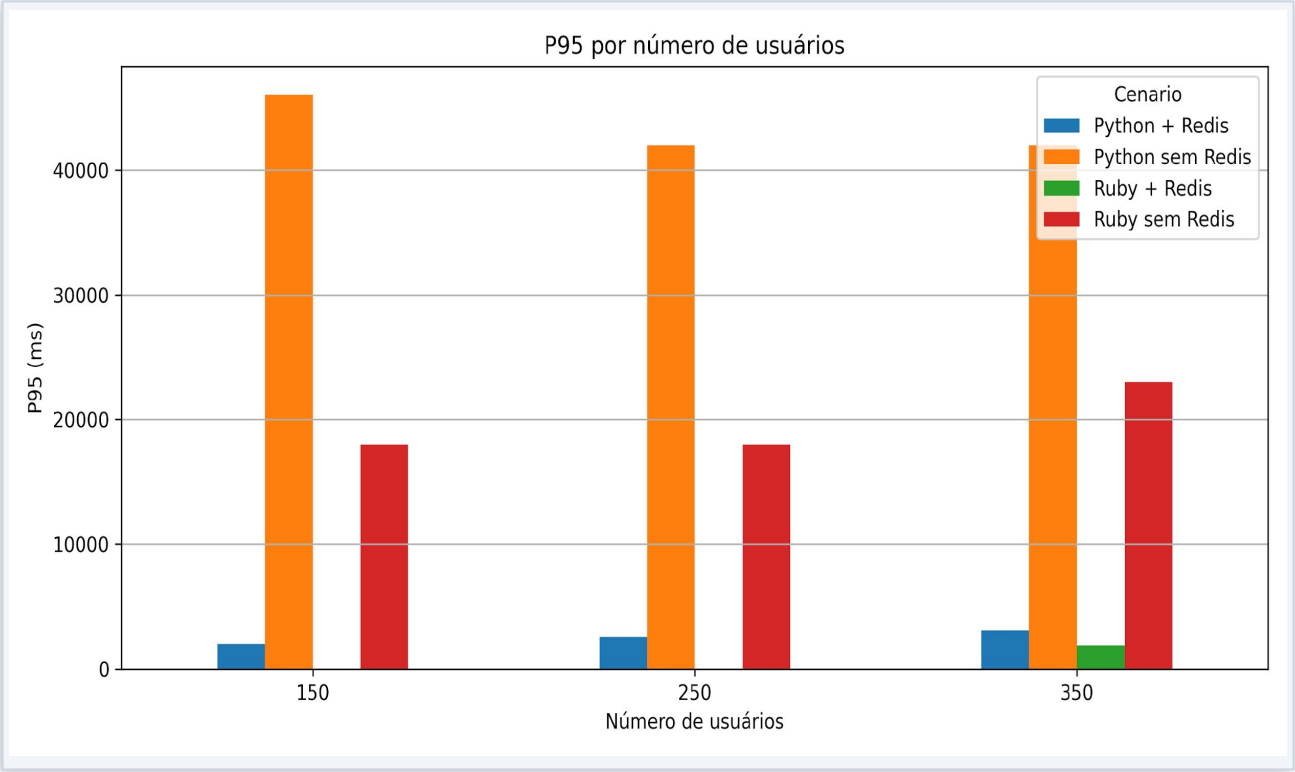
Tempo Médio de Resposta — Todos os Cenários por Nº de Usuários



Python sem Redis apresenta tempo médio entre 22.000 e 25.000 ms em todos os níveis de carga, dominando completamente o gráfico. Python + Redis e Ruby + Redis ficam próximos de zero na escala, confirmando que o cache elimina praticamente toda a latência independente da quantidade de usuários.

Python sem Redis domina com ~25.000 ms em todos os níveis. Python + Redis mantém ~420 ms estável.

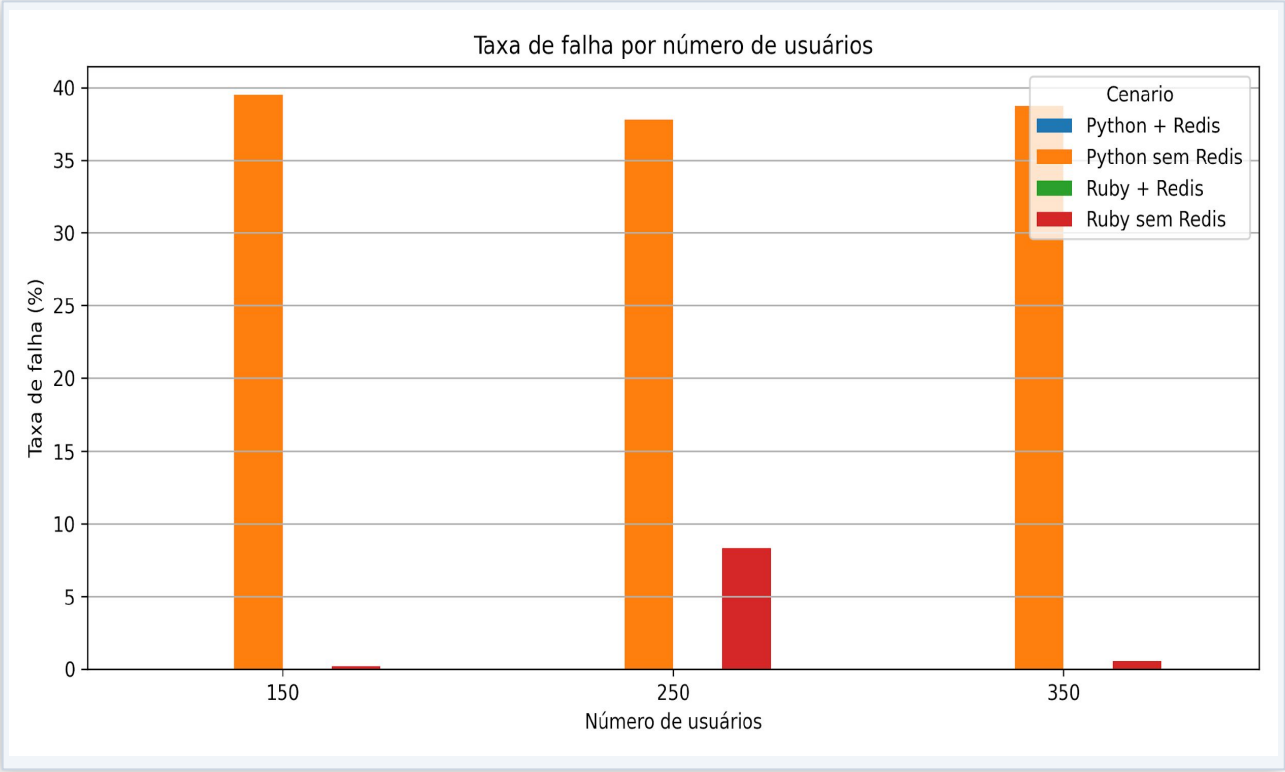
P95 de Tempo de Resposta — Todos os Cenários por Nº de Usuários



O P95 do Python sem Redis ultrapassa 40.000 ms já na carga leve, indicando que 5% das requisições mais lentas sofrem esperas extremas. Ruby sem Redis também apresenta P95 elevado (~18.000–23.000 ms), enquanto os cenários com Redis se mantêm abaixo de 3.200 ms em todos os níveis.

Python sem Redis atinge P95 de 46.000 ms na carga leve. Ruby + Redis permanece abaixo de 2.000 ms.

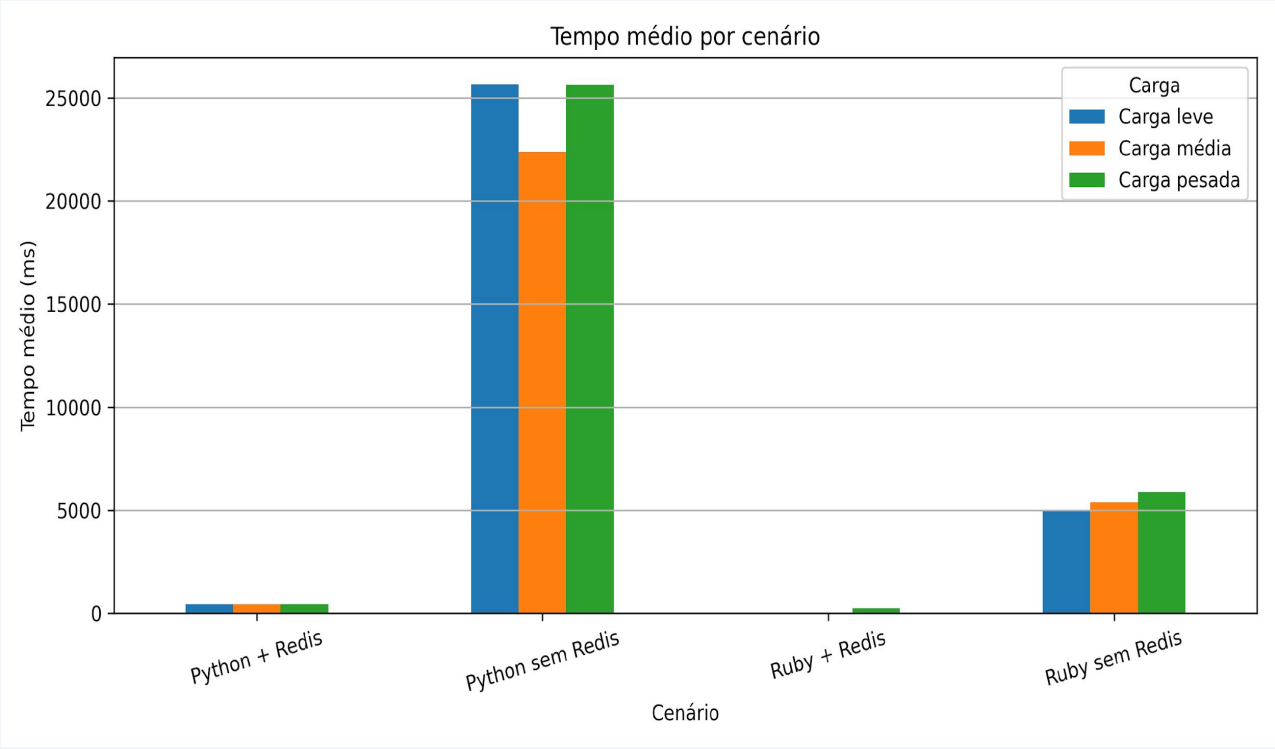
Taxa de Falha — Todos os Cenários por Nº de Usuários



Python sem Redis registra cerca de 39% de falhas em todos os níveis de carga — o servidor simplesmente não consegue atender as requisições a tempo. Ruby sem Redis apresenta falhas pontuais de até 8,3% na carga média. Todos os cenários com Redis mantêm 0% de falha .

Python sem Redis registra ~39% de falhas. Python + Redis e Ruby + Redis mantêm 0%.

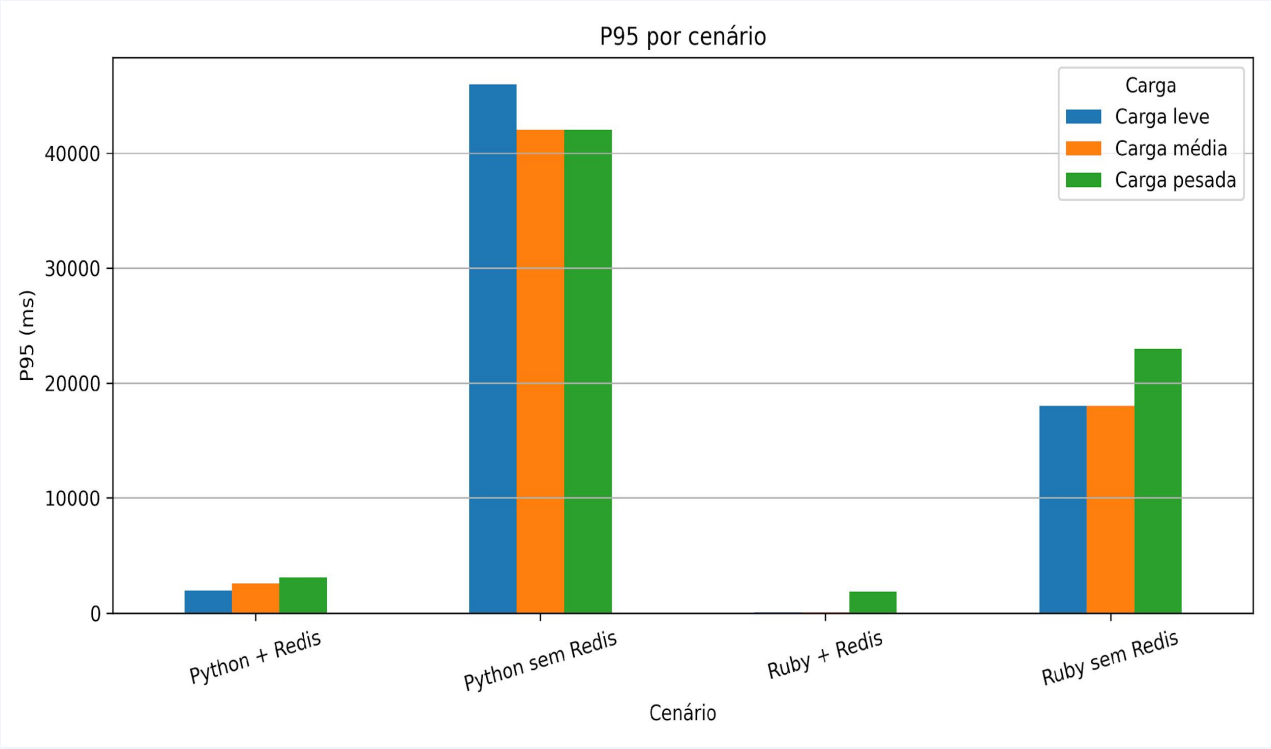
Tempo Médio por Cenário e Nível de Carga



A comparação direta entre cenários deixa evidente o impacto do Redis: Python + Redis fica ~60× mais rápido que Python sem Redis. Ruby sem Redis, apesar de mais lento que os cenários com cache, apresenta tempo médio bem inferior ao Python sem Redis (~5.000 ms vs ~23.000 ms).

Comparação direta: Redis reduz o tempo médio drasticamente em ambas as linguagens.

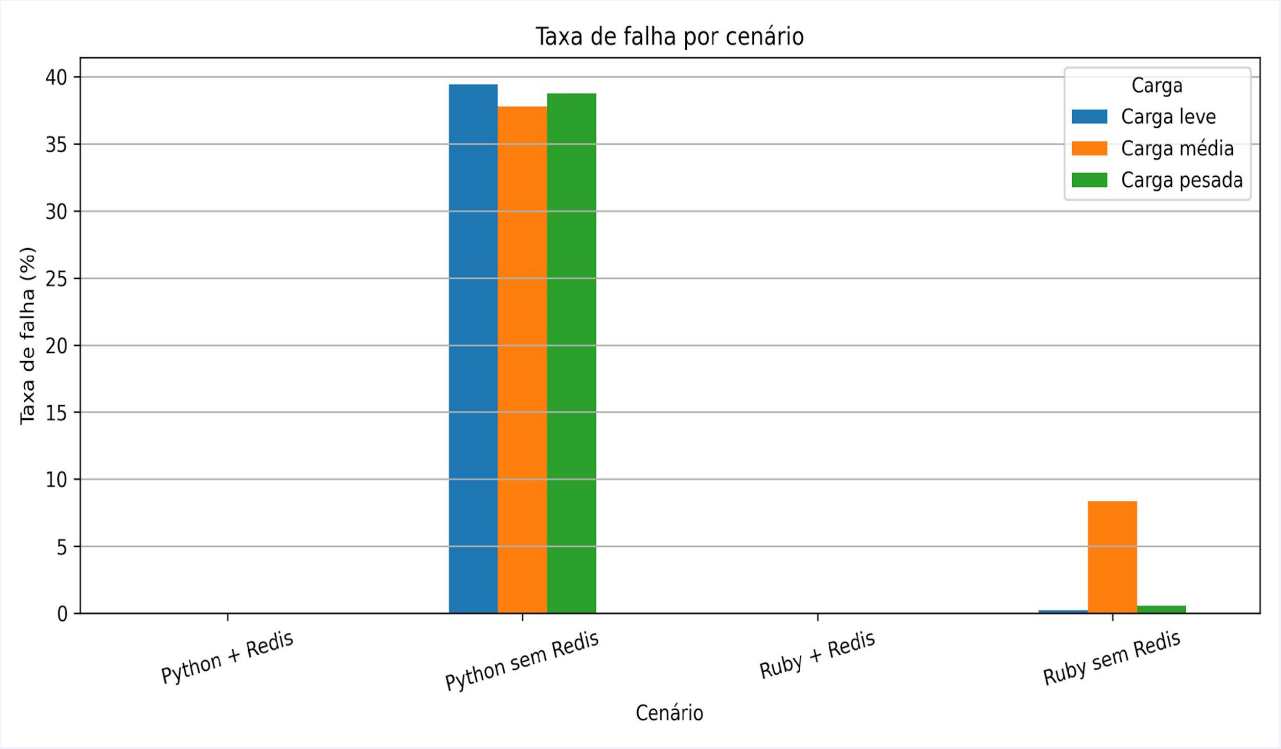
P95 por Cenário e Nível de Carga



Os cenários sem Redis concentram os maiores P95 independente da linguagem, confirmando que a ausência de cache é o fator determinante para latências extremas. O P95 do Python sem Redis na carga leve supera 45.000 ms — mais de 14× o P95 do Ruby sem Redis no mesmo nível.

Cenários sem Redis concentram P95 muito alto, independente da linguagem.

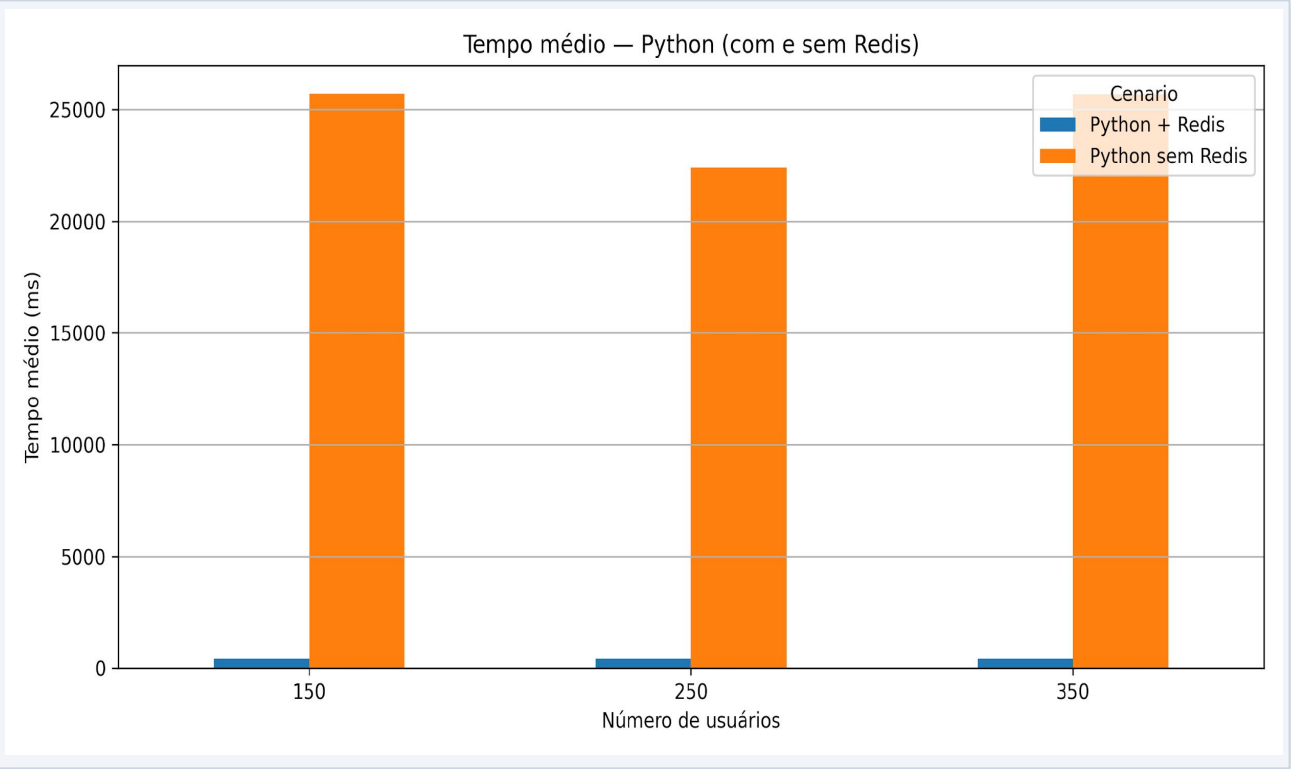
Taxa de Falha por Cenário e Nível de Carga



As falhas se concentram exclusivamente nos cenários sem Redis. Python sem Redis é consistentemente o pior, com ~39% em todas as cargas. Ruby sem Redis apresenta comportamento irregular — 0,3% na leve, pico de 8,3% na média, caindo para 0,6% na pesada — sugerindo instabilidade no serviço durante os testes.

Falhas concentradas exclusivamente nos cenários sem Redis.

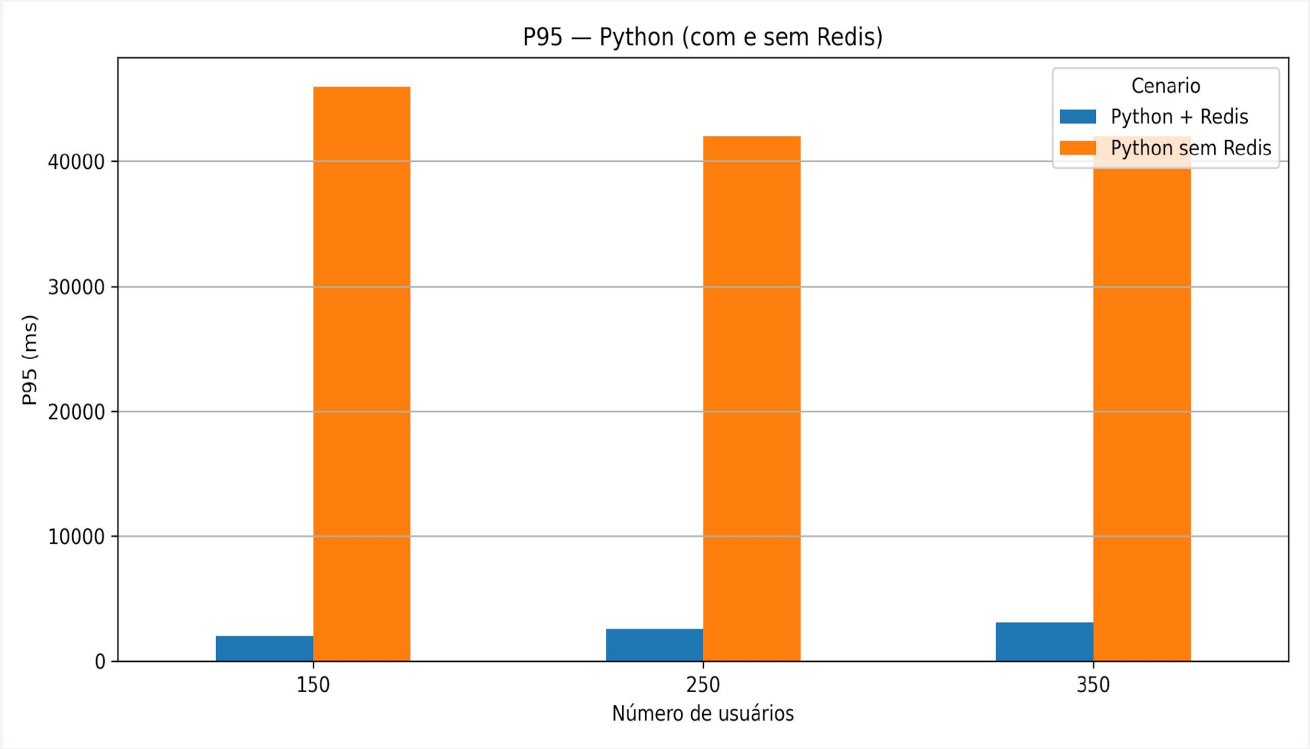
Python — Tempo Médio: Com vs Sem Redis



Com Redis, o serviço Python mantém ~420 ms constante nos três níveis de carga, sem degradação perceptível com o aumento de usuários. Sem Redis, o tempo médio oscila entre 22.000 e 25.000 ms — a diferença é de aproximadamente 55 a 60 vezes.

Com Redis: ~420 ms constante. Sem Redis: ~25.000 ms — diferença de ~60×.

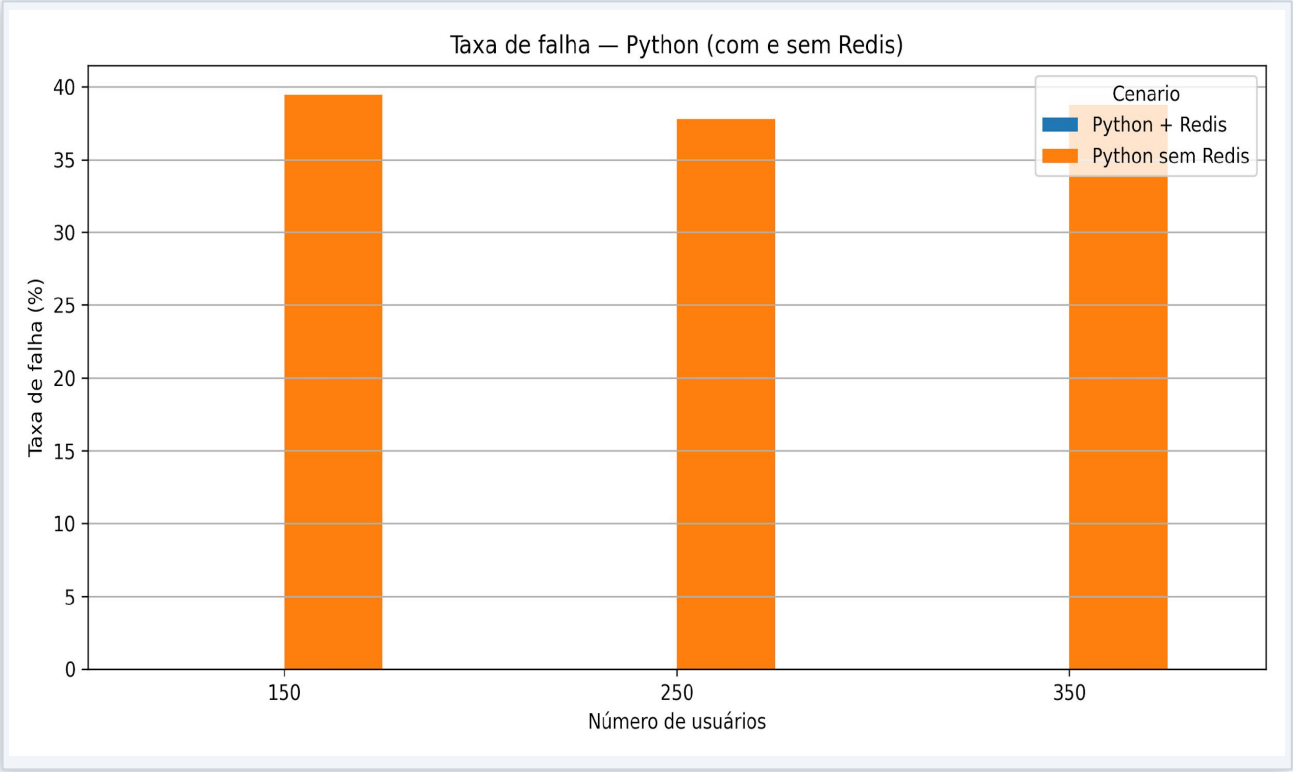
Python — P95: Com vs Sem Redis



O P95 do Python com Redis fica abaixo de 3.200 ms em todos os cenários. Sem Redis, o P95 parte de 46.000 ms na carga leve e se mantém acima de 40.000 ms nos demais níveis, demonstrando que as requisições mais lentas são severamente impactadas.

P95 sem Redis ultrapassa 40.000 ms. Com Redis permanece abaixo de 3.200 ms.

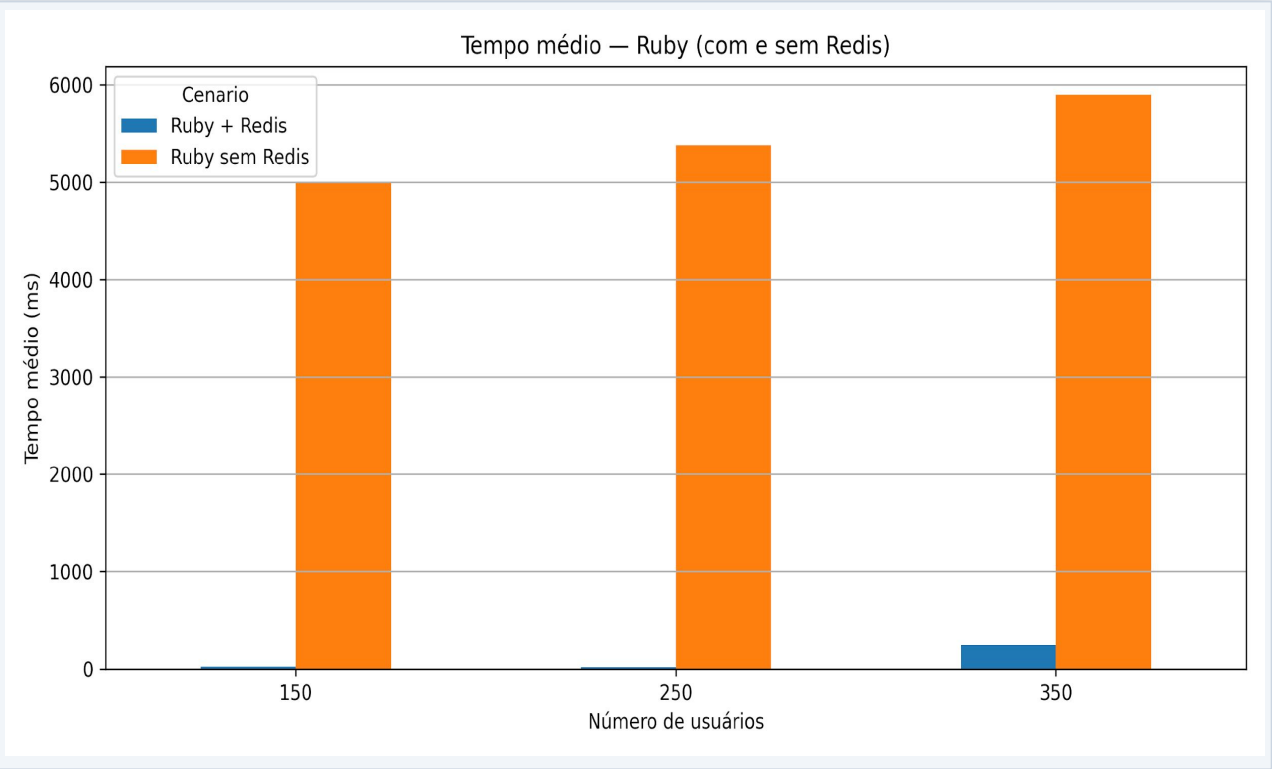
Python — Taxa de Falha: Com vs Sem Redis



Python com Redis: 0% de falhas em todos os testes. Python sem Redis: ~39% de falhas constantes, independente da carga. O cache não apenas reduz a latência — ele é o que viabiliza o serviço Python em produção.

Sem Redis: ~39% de falhas em todos os níveis. Com Redis: 0% em todos os testes.

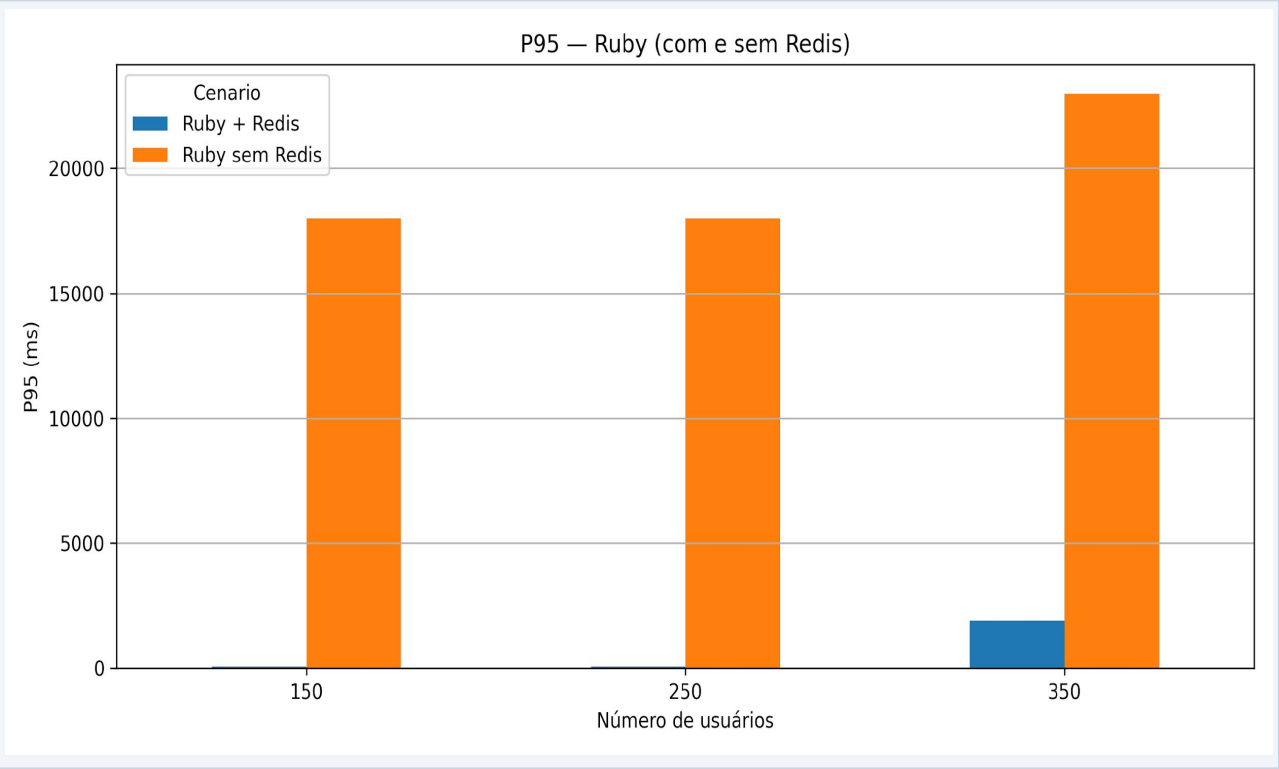
Ruby — Tempo Médio: Com vs Sem Redis



Ruby sem Redis apresenta tempo médio crescendo de ~5.000 ms (leve) para ~6.000 ms (pesado). Ruby com Redis fica abaixo de 30 ms nas cargas leve e média, subindo para ~250 ms na carga pesada — comportamento compatível com saturação de conexões no container.

Ruby + Redis: inferior a 250 ms. Ruby sem Redis: ~5.000–6.000 ms.

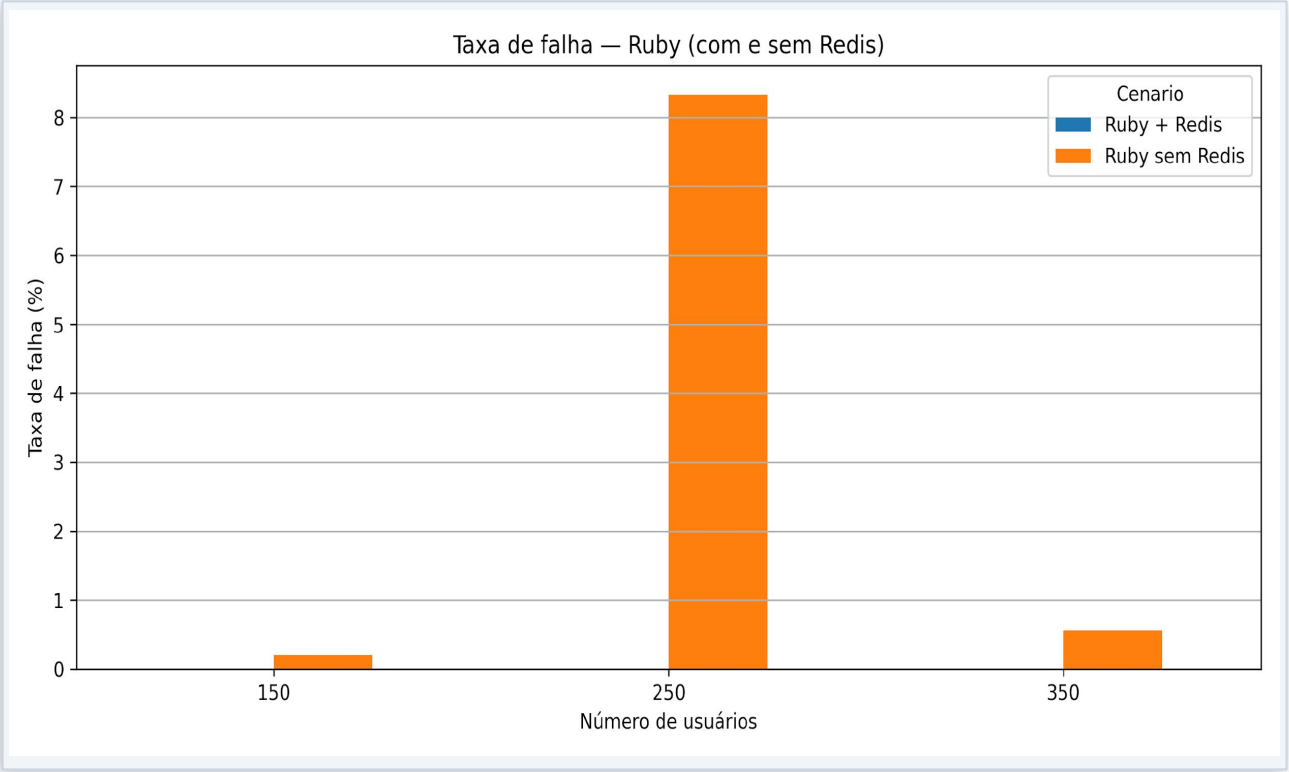
Ruby — P95: Com vs Sem Redis



O P95 do Ruby sem Redis chega a 23.000 ms na carga pesada. Ruby com Redis mantém P95 muito baixo nas cargas leve e média, com elevação para ~1.700 ms na carga pesada — ainda dentro de limites aceitáveis, mas indicando pressão crescente.

Ruby sem Redis P95 chega a 23.000 ms na carga pesada.

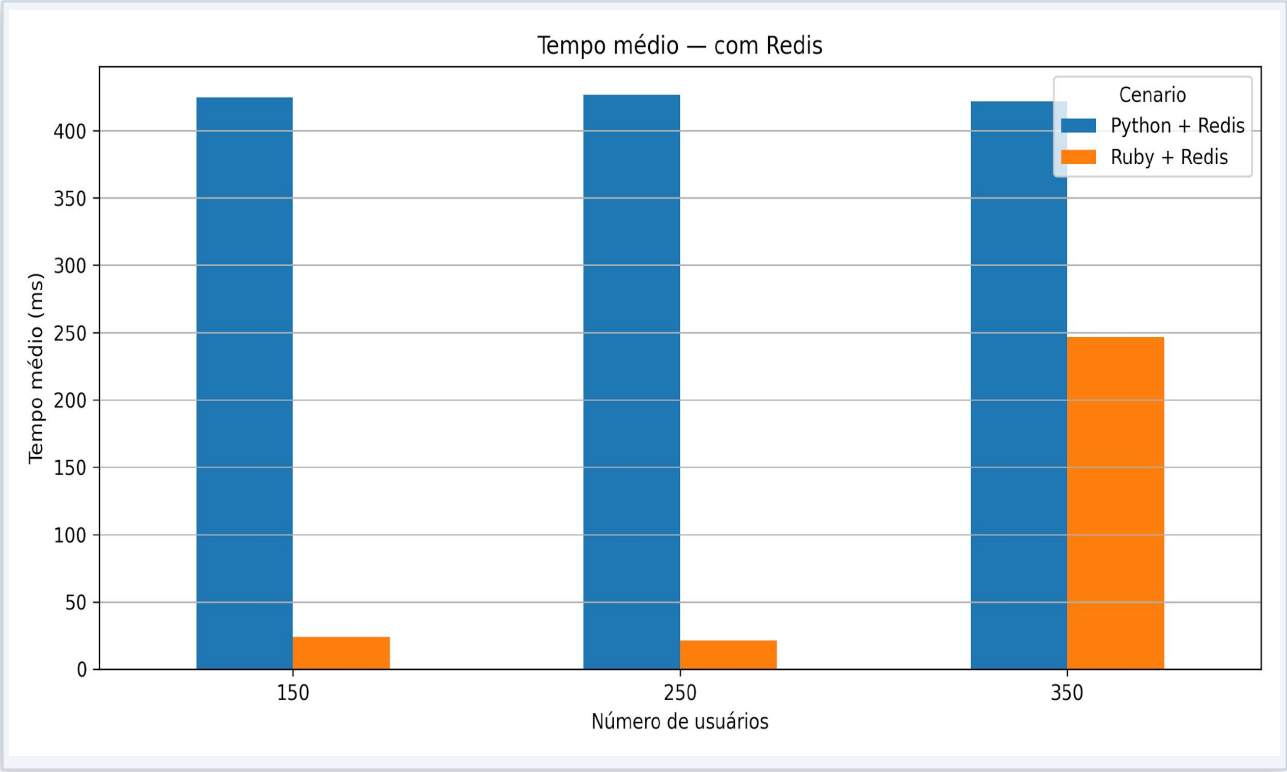
Ruby — Taxa de Falha: Com vs Sem Redis



Ruby com Redis: 0% de falhas. Ruby sem Redis: pico de 8,3% na carga média, com comportamento irregular entre os níveis. A inconsistência nas falhas do Ruby sem Redis pode indicar instabilidade no serviço durante os testes realizados.

Ruby sem Redis apresenta até 8,3% de falhas (carga média). Ruby + Redis: 0%.

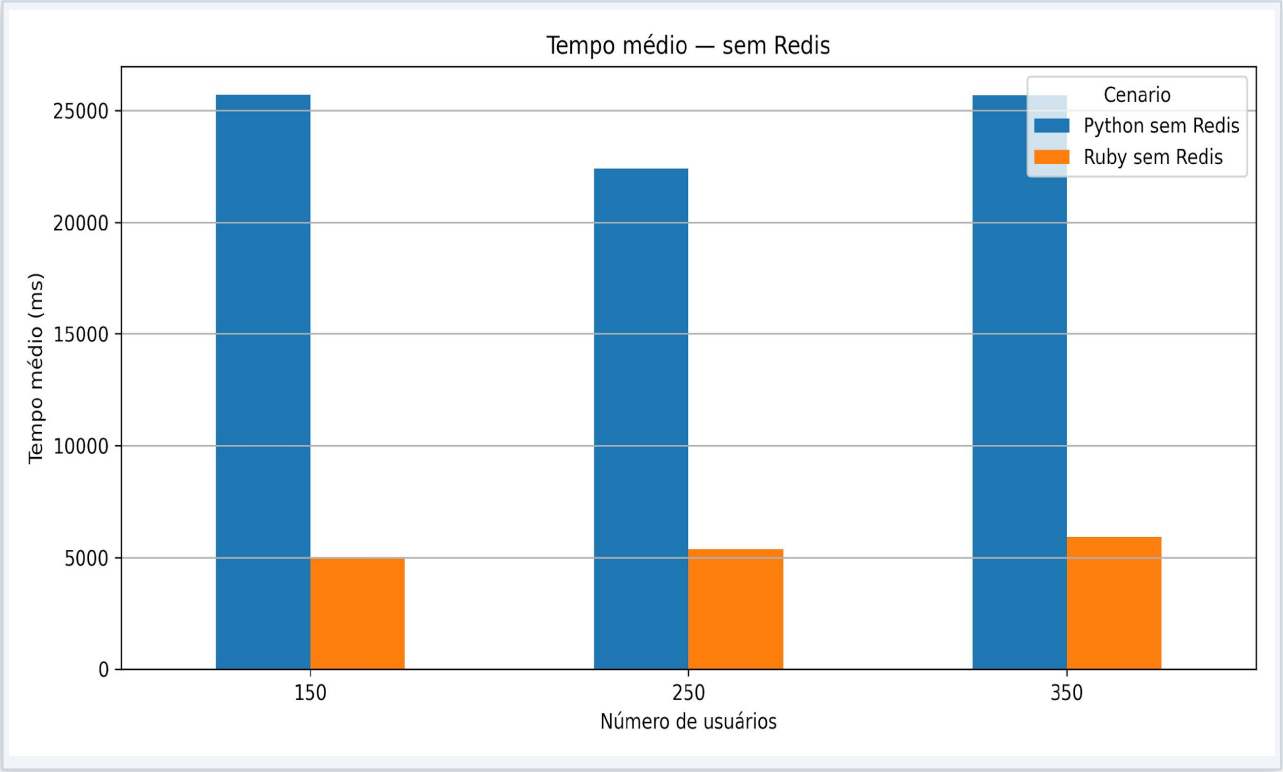
Com Redis — Python vs Ruby (Tempo Médio)



Com cache ativo, Ruby é consistentemente mais rápido que Python nas cargas leve e média (~25 ms vs ~420 ms). Na carga pesada, Ruby sobe para ~250 ms enquanto Python se mantém estável em ~420 ms. A diferença pode estar relacionada ao comportamento suspeito do Ruby + Redis apontado anteriormente.

Ruby + Redis consistentemente mais rápido que Python + Redis nos primeiros níveis de carga.

Sem Redis — Python vs Ruby (Tempo Médio)



Sem cache, Python é significativamente mais lento que Ruby em todos os níveis (~23.000 ms vs ~5.500 ms). Ambos os cenários são inviáveis para uso em produção sob carga, mas o Ruby demonstra maior capacidade de resposta mesmo sem o benefício do cache.

Python sem Redis é ~4× mais lento que Ruby sem Redis. Ambos inaceitáveis para produção.

Conclusões

Redis é indispensável

O cache elimina praticamente toda a latência e as falhas. Sem Redis, ambas as APIs ficam inutilizáveis sob carga.

Linguagem é fator secundário

A presença ou ausência de cache impacta ordens de grandeza a mais do que a escolha de linguagem.